

Encadrant :
PASDELOUP Bastien
bastien.pasdeloup@telecom-bretagne.eu

Membres de l'équipe de projet :
ECHRAIBI Amine
amine.echraibi@telecom-bretagne.eu

NDZAMBA Gildas Louis
louis.ndzamba@telecom-bretagne.eu

MOUBAKIR Lotfi
lotfi.moubakir@telecom-bretagne.eu



Déploiement de logiciel de machine virtuelle et exécution de programme en environnement contrôlé.

Rapport de projet

Formation ingénieur généraliste première année

Projet développement

Année scolaire 2015-2016

Date : 04/06/2016 version 2

Projet numéro 2.



Sommaire

1. INTRODUCTION	3
2. STRUCTURE DE LA PLATE-FORME	4
2.1 DESCRIPTION DE L'EXISTANT :	4
2.2 EXEMPLES DE FAILLES :	5
3. SOLUTION ENVISAGEABLE	5
3.1 UTILISATION DES MACHINES VIRTUELLES	5
3.2 MISE AU POINT DES RESEAUX SEPARES	5
4. OUTIL DE DEVELOPPEMENT	7
4.1 COMPARAISON DE LOGICIEL DE MACHINE VIRTUELLE.....	7
4.2 PRESENTATION DU LOGICIEL VIRTUALBOX	8
4.2.1 Les modes réseaux	8
4.2.2 Virtual Box et la gestion des machines virtuelles	9
5. ETAPES DE DEVELOPPEMENTS	9
5.1 CREATION ET CONFIGURATION DE MACHINE VIRTUELLE.....	9
5.2 PROGRAMME DE DETECTION DU FICHIER.....	10
5.3 CREATION DES RESEAUX VIRTUELS SEPARES	11
5.3.1 Création des réseaux :	11
5.3.2 Tests de séparation :	12
5.4 PROGRAMME PRINCIPAL	13
5.4.1 Démarrage des machines virtuelles :	13
5.4.2 Envoi du fichier python :	13
5.4.3 Etablissement de la communication TCP :	14
6. PORTEES ET LIMITES DE LA SECURISATION	17
6.1 COMPARAISON ENTRE L'ANCIEN SYSTEME ET LE NOUVEAU.....	17
6.2 LIMITES DU SYSTEME DEVELOPPE	18
7. DIAGRAMME DE GANT	18
7.1 DIAGRAMME DE GANT	18
7.1.1 Diagramme de Gant initial :	18
7.1.2 Diagramme de Gant final :	19
7.2 COMPARAISON ENTRE DIAGRAMME DE GANT PREVISIONNEL ET LE DEROULEMENT DU PROJET	19
8. CONCLUSION	20
9. GLOSSAIRE	21
10. REFERENCES	22

Résumé

Dans le cadre de notre projet, nous sommes amenés à sécuriser la plateforme Pyrat. Cette plateforme a été développée pour rendre le module de tronc commun intitulé renforcement des acquis en algorithmique et programmation plus accessible aux étudiants. Dans ce module, les étudiants sont amenés à créer des scripts python pour contrôler un rat dans un labyrinthe, l'objectif étant de collecter le plus de pièces possibles dans le labyrinthe.

Les étudiants étaient capables de tricher à cause de l'architecture non-sécurisée de la plateforme. En effet, les deux programmes des joueurs tournant sur la même machine, ils partagent les mêmes ressources. Un joueur peut donc facilement tuer les processus de l'autre joueur ou envoyer des données erronées en prétendant être l'autre joueur par exemple.

Dans le but d'empêcher que cela n'arrive, notre travail a consisté à créer et configurer deux machines virtuelles où tourneront le programme de chaque joueur. Les programmes des joueurs seront exécutés dans des environnements contrôlés, de ressources équitables et complètement isolés l'un de l'autre. Puis, pour communiquer les résultats de calcul, les deux machines virtuelles seront attachées à la machine réelle par des sockets TCP chacune sur son réseau totalement séparé l'un de l'autre.

Ecrit par Louis Gildas
Relecture : Lotfi Moubakir

1. INTRODUCTION

Dans le cadre de l'enseignement de première année de formation d'ingénieur, le corps enseignant a mis en place un jeu compétitif éducatif pour le module de tronc commun : renforcement des acquis en algorithmique et programmation. Dans ce module, les étudiants sont amenés à écrire des scripts python qui guident un rat dans le but de collecter des pièces dans un labyrinthe : PYRAT.[1]

A la fin du module, chaque groupe d'étudiants a amené son programme pour le tournoi PYRAT dans le but de tester son efficacité par rapport aux autres. Au cours de ce tournoi certain étudiant ont triché en se servant des failles du système PYRAT. En effet, les deux programmes fournis par les étudiants tournent sur la même machine donc ont accès aux ressources de la machine.

Le problème auquel notre projet répond est donc le suivant : Comment rendre la plate-forme PYRAT plus sécurisée contre ce type de piratage ?

Nous allons pour répondre à cette question étudier des solutions à base de machines virtuelles et proposer un code permettant de lancer des programmes en environnement contrôlé. Des fichiers vont être envoyé à chaque machine virtuelle par un programme principal, ensuite les machines virtuelles exécuteront ces deux fichiers puis renverront le résultat à la machine réelle, puis la machine réelle sauvegardera le résultat d'exécution.

Dans un premier temps, nous étudierons la structure existante non sécurisée. Dans un deuxième temps, nous exposerons une solution succincte au problème que nous avons posé. Dans un troisième temps, nous décrirons l'outil de développement sur lequel nous nous sommes appuyés pour la solution envisagée et décrirons les étapes détaillées de la solution envisagée. Enfin, nous montrerons les tests et résultats de la sécurisation et les limites de cette solution, ce qui nous permettra de conclure.

Ecrit par Lotfi Moubakir
Relecture : Amine Echraibi

2. STRUCTURE DE LA PLATE-FORME

2.1 DESCRIPTION DE L'EXISTANT :

La plate-forme PYRAT existante est exécutée sur un ordinateur. Cela est fait à l'aide d'un terminal Linux avec la commande suivante :
`./executables/matchAIAI.sh joueur1.py joueur2.py`

Le programme `matchAIAI.sh` prend donc en entrée deux programmes python `joueur1.py` qui correspond au premier joueur et `joueur2.py` qui correspond au deuxième joueur. Il s'agit de deux programmes qui tournent en parallèle et font leurs calculs sur des graphes puis qui envoient le résultat à la plate-forme pour contrôler les petits points rouges et verts. La figure 1 donne une vision globale de l'ancien système.

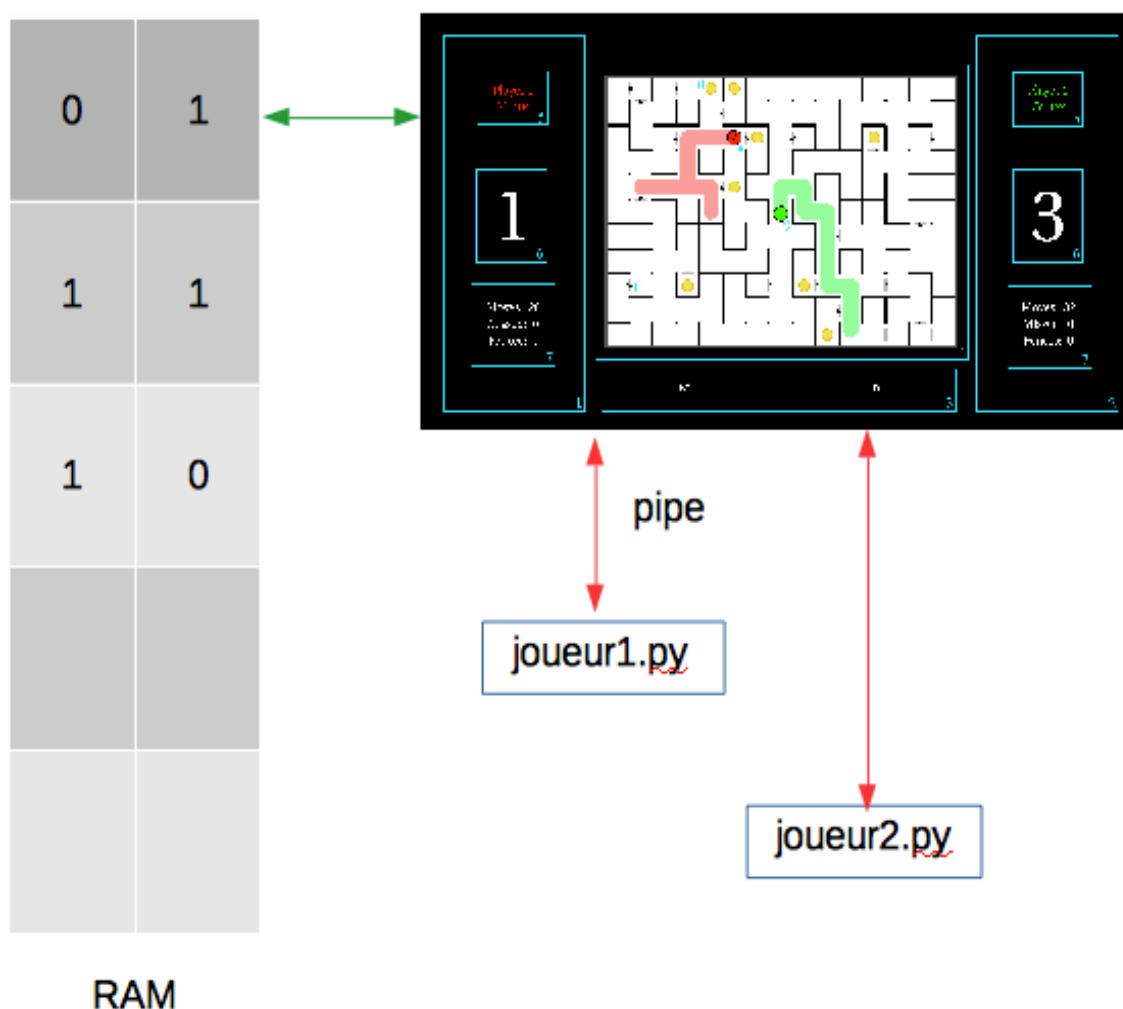


Figure 1 : Fonctionnement de la plateforme Pyrat.

2.2 EXEMPLES DE FAILLES :

L'inconvénient de cette structure est l'exécution non contrôlée des deux programmes joueur1.py et joueur2.py sur la machine. Donc ils peuvent interagir avec les ressources du système et avec les autres processus qui tournent en parallèle sur la machine. De ce fait, le programme du joueur 1 peut par exemple effectuer des kill sur les processus générés par le programme du joueur 2. Cela étant un simple exemple, ils peuvent envoyer de fausses commandes en prétendant être le deuxième joueur. On peut aussi allouer plus de ressources à un des programmes (mémoire, priorité pendant l'exécution ... etc).

Nous voyons donc que le système est non adapté pour certaines attaques et allons proposer une solution.

Ecrit par Amine Echraibi
Relecture : Louis Gildas

3. SOLUTION ENVISAGEABLE

3.1 UTILISATION DES MACHINES VIRTUELLES

La virtualisation est une des solutions envisageables afin de sécuriser la plate-forme Pyrat. En effet, les machines virtuelles sont des émulations de différents appareils informatiques et de leurs ressources matérielles, le logiciel de simulation d'une machine virtuelle fait en sorte de cloner l'environnement d'une machine réelle de manière à ce que l'on puisse exécuter des programmes dans les mêmes conditions que celles de la machine simulée.

Les machines virtuelles présentent un grand avantage car elles nous permettent de s'abstraire des caractéristiques de la machine réelle et permet d'utiliser des environnements séparés et des ressources équitables. En plus ils nous présentent la possibilité d'isoler l'exécution de programmes (Virus, malware ...etc.) et monitorer en temps réel l'effet de ces programmes sur la machine virtuelle, ainsi les machines virtuelles sont des outils efficaces pour établir des protocoles de sécurité.

3.2 MISE AU POINT DES RESEAUX SEPARES

L'utilisation simple d'une machine virtuelle ne suffit pas afin de garantir une bonne sécurité. Pour ce faire, nous devons créer deux

environnements. Les deux machines virtuelles se trouveront sur deux réseaux virtuels LAN séparés.

Elles communiqueront uniquement avec la machine hôte. Le principe de notre solution est le suivant : les programmes joueur1.py et joueur2.py sont fournis par les joueurs, puis la machine réelle démarre les deux machines virtuelles chacune sur son réseau, ensuite elle envoie chaque fichier python à une machine virtuelle en utilisant son réseau puis les machines virtuelles exécutent les deux fichiers, et ils renvoient le résultat à la machine réelle par communication TCP.

Ainsi, l'exécution de chaque programme sera faite dans un environnement contrôlé et complètement isolé de l'autre, ce qui minimise les méthodes de hacks. La figure 2 donne une vision globale du nouveau système.

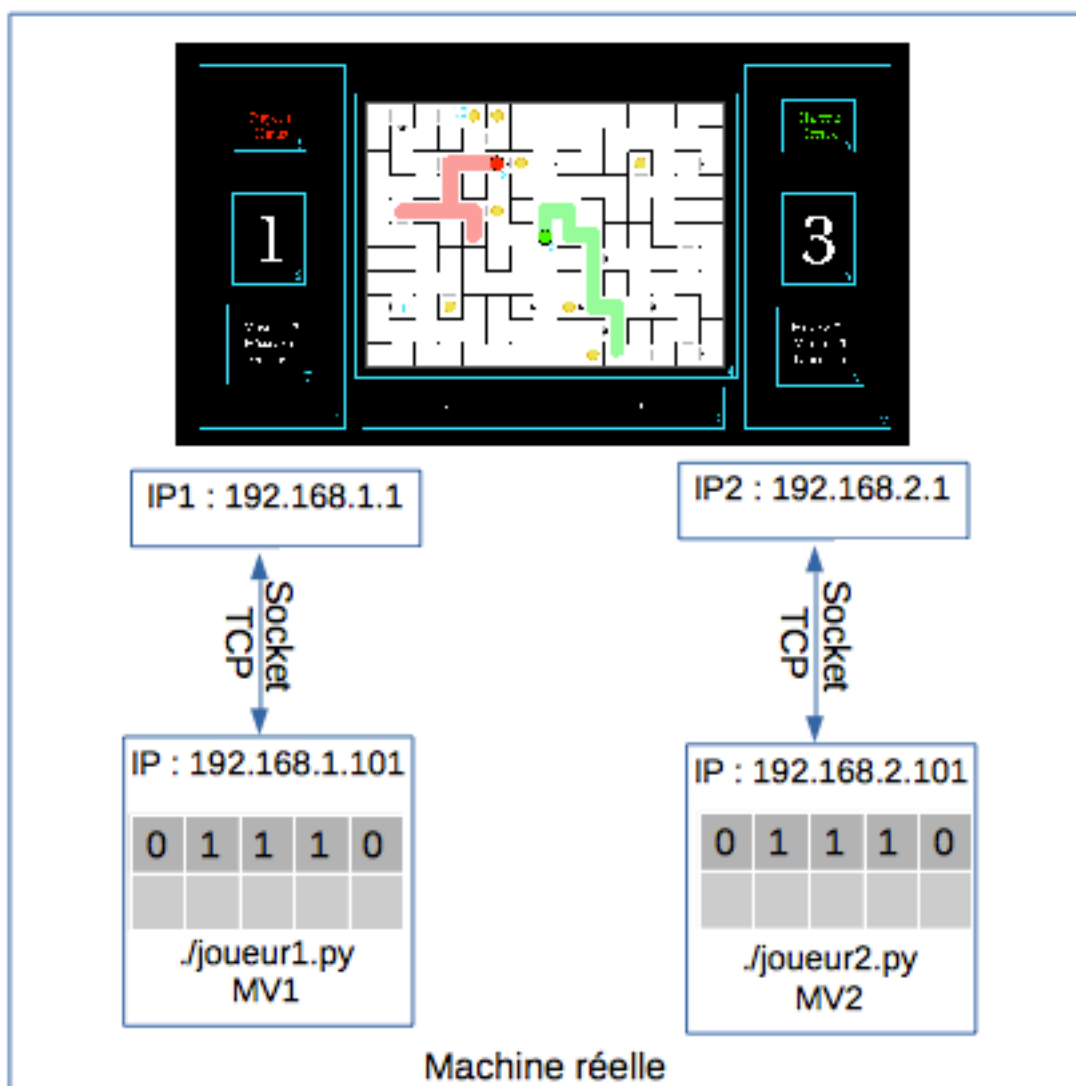


Figure 2 : Plateforme Pyrat après sécurisation.

Nous venons d'exposer dans les grandes lignes comment fonctionne la solution. Nous allons désormais décrire précisément ce qui compose la solution.

Ecrit par Louis Gildas
Relecture : Amine Echraibi

4. OUTIL DE DEVELOPPEMENT

4.1 COMPARAISON DE LOGICIEL DE MACHINE VIRTUELLE

Le système d'exploitation hôte (et imposé par l'encadrant de notre projet) est un système Linux donc on ne peut pas utiliser le logiciel Windows Virtual PC, ce qui nous laisse trois logiciels célèbres de

simulation de système d'exploitation Linux : Qemu, Virtual Box et VMWare.

QEMU nous permet de simuler des machines virtuelles sur une machine hôte sans avoir l'accès administratif à cette machine, ce qui présente un désavantage en termes de sécurité.

Il nous est imposé par notre projet d'utiliser un logiciel gratuit, ce qui exclut VMWare. Il ne nous reste donc comme choix de logiciel Virtual Box.

4.2 PRESENTATION DU LOGICIEL VIRTUALBOX

Virtual Box est le logiciel que nous avons retenu, nous allons donc étudier ses fonctionnalités en détail.

4.2.1 Les modes réseaux

- Non attaché : équivaut à une interface Ethernet sur laquelle aucun câble n'est connecté
- NAT : Il permet de configurer la machine pour partager les adresses IP et MAC de l'hôte. Cela implique que la machine virtuelle et l'hôte partagent une seule et même adresse réseau visible depuis l'extérieur.
- Adaptateur réseau hôte : Le réseau ne possède qu'une existence interne, sans interconnexion avec l'extérieur. Virtual Box ne fournit pas de serveur DHCP sur ce réseau privé virtuel. Il faudra donc configurer les adresses IP à la main ou bien utiliser un serveur DHCP sur la machine hôte (machine physique).
- Interne : Les machines virtuelles sont reliées entre elles mais isolées de l'extérieur
- Bridge (ou "accès par pont") : Il permet de configurer la machine virtuelle pour qu'elle ait une identité unique sur le réseau, et qu'elle soit distincte de son hôte et non liée à celui-ci. Son interface réseau dispose d'une adresse IP et d'une adresse MAC différentes de celles de la machine hôte. Avec ce mode, les machines virtuelles peuvent communiquer totalement entre elles vers l'extérieur via la machine hôte.

4.2.2 Virtual Box et la gestion des machines virtuelles

Virtual Box possède des commandes terminales qui permet de contrôler les machines virtuelles. De plus, il y a une interface graphique pour pouvoir créer et configurer des machines virtuelles contrairement à d'autres logiciels tels Qemu qui n'utilisent que des commandes terminales pour manipuler les machines virtuelles : on dit que Virtual Box est « userfriendly ».

Virtual Box satisfait à toutes les contraintes imposées et possède les modes réseaux qui nous intéressent, sa gestion des machines virtuelles est intéressante en cela qu'elle est plus simple d'utilisation avec son interface graphique, c'est pourquoi nous l'avons retenu.

Ecrit par Lotfi Moubakir
Relecture : Amine Echraibi

5. ETAPES DE DEVELOPPEMENTS

5.1 CREATION ET CONFIGURATION DE MACHINE VIRTUELLE

Plusieurs sites internet nous montrent comment créer une machine virtuelle à l'aide de virtualbox. Il existe un de ces tutoriels dans lequel la création de machine virtuelle est détaillée. Ce qu'il faut retenir de cette création, c'est le choix de la mémoire RAM lors de l'installation, un choix important car cette mémoire sera soustraite à la mémoire vive de notre pc. La mémoire RAM disponible physiquement (par exemple 4 Go) devra non seulement suffire pour supporter le système d'exploitation principal (sur lequel est installé Virtual box) mais également pour vos machines virtuelles. Dans le cas d'un ordinateur doté de peu de RAM, vous allez sentir très rapidement un goulot d'étranglement. Pendant la configuration de la RAM, il faudra donc prendre en compte le fait qu'il y aura deux machines virtuelles avec les mêmes ressources.[2]

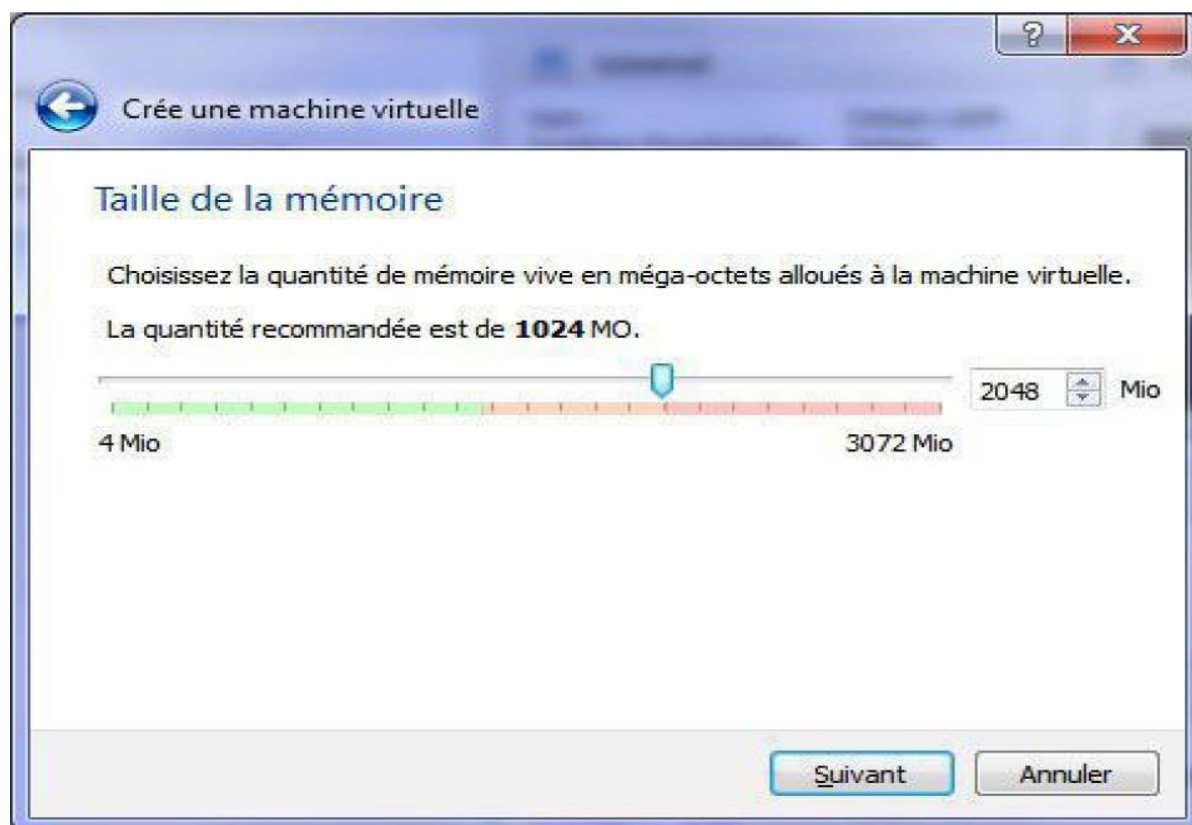


Figure 3: allocation de mémoire à l'aide de virtualbox

Après allocation de la mémoire, on continue suivant la configuration recommandée par le logiciel VirtualBox. Puis, nous installons l'Os Ubuntu sur la machine virtuelle. Après installation de l'os, il faut configurer le réseau de la machine et mettre au point le système qui va détecter le fichier python envoyé par la machine réelle. On ajoute dans le fichier `bashrc` de l'os Ubuntu la commande qui démarre le programme de détection (cf partie 5.2). Ensuite, le fichier `bashrc` est exécuté après chaque démarrage de la machine virtuelle, donc le lancement de programme de détection s'effectue après chaque démarrage.

Pour configurer la deuxième machine, il suffit de cloner la première et reconfigurer le réseau.[3]

5.2 PROGRAMME DE DETECTION DU FICHIER

Après création et configuration des machines virtuelles, chacune de ces machines gardera un état fixe à laquelle elle va être réinitialisée pendant chaque démarrage. Elles doivent être capables de détecter l'arrivée du fichier python de chaque joueur. Cela est établi en gardant

un dossier de réception vide dans l'état initial de la machine et en lançant automatiquement après le démarrage de chaque machine l'exécution d'un programme de détection. Ce programme teste si le fichier existe et s'il existe, il l'exécute. Sinon, il attend une petite période de temps (2 secondes) et puis il fait un test d'existence une deuxième fois. Le schéma suivant décrit son fonctionnement :[4]

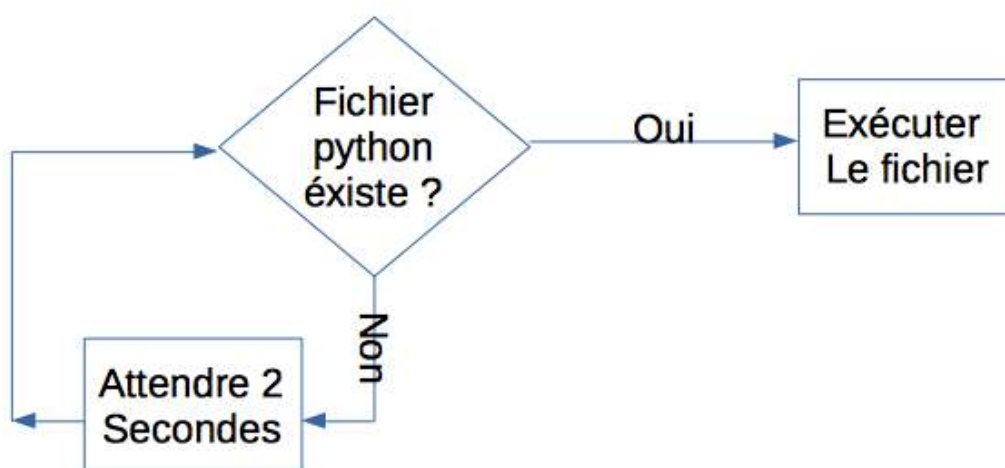


Figure 5 : Programme de détection de fichier

Ecrit par Amine Echraibi
Relecture : Louis Gildas

5.3 CREATION DES RESEAUX VIRTUELS SEPARES

5.3.1 Création des réseaux :

VirtualBox nous permet de créer des interfaces réseaux en plusieurs modes comme mentionné dans la partie 4.2. Le mode adapté à notre application est le mode Host-Only. Ce mode nous permet de créer deux interfaces réseaux virtuelles, chacune étant attachée à une machine virtuelle. Pour ce faire, il faut définir l'adressage qui convient avec pour but de séparer les deux réseaux (cf figure 6). La configuration du deuxième réseau se fait de la même manière, il suffit pour cela de remplacer la plage 56 par 57.[8]



Figure 6 : Configuration du premier réseau

5.3.2 Tests de séparation :

Pour tester la séparation des réseaux, nous avons essayé de nous connecter à distance à une des machine depuis l'autre machine, puis nous avons aussi essayé d'effectuer des captures et des analyses de réseaux depuis une machine virtuelle pour voir s'il est possible de détecter ou d'infiltrer le réseau de l'autre machine, ce qui s'avère inutile. En effet, le réseau n'est pas atteignable, ce qui montre que les deux réseaux sont bien séparés. La figure 7 montre les résultats de ces tests.

```

vm1@vm1-VirtualBox: ~
vm1@vm1-VirtualBox:~$ ssh -X vm2@192.168.57.102
ssh: connect to host 192.168.57.102 port 22: Network is unreachable
vm1@vm1-VirtualBox:~$ arp -a
? (192.168.56.1) at 0a:00:27:00:00:00 [ether] on enp0s3
vm1@vm1-VirtualBox:~$ sudo tcpdump -v
[sudo] password for vm1:
tcpdump: listening on enp0s3, link-type EN10MB (Ethernet), capture size 262144 bytes

15:54:15.380917 IP (tos 0x0, ttl 255, id 2346, offset 0, flags [DF], proto UDP (17), length 73)
    192.168.56.100.mdns > 224.0.0.251.mdns: 0 [2q] PTR (QM)? _ipps._tcp.local. PTR (QM)? _ipp._tcp.local. (45)
15:54:16.304136 IP6 (flowlabel 0x805c3, hlim 255, next-header UDP (17) payload length: 53) fe80::c77b:dd00:b08d:81cf.mdns > ff02::fb.mdns: [bad udp cksum 0xd59e -> 0xfede!] 0 [2q] PTR (QM)? _ipps._tcp.local. PTR (QM)? _ipp._tcp.local. (45)

```

Figure 7 : Résultats des tests de séparation.

Ecrit par Louis Gildas
Relecture : Lotfi Moubakir

5.4 PROGRAMME PRINCIPAL

Après la création des réseaux séparés, on a écrit le programme principal. Ce programme sera capable de démarrer et réinitialiser les deux machines virtuelles et lancer les cotés serveurs de la communication TCP sur les deux réseaux et puis envoyer les fichiers pythons aux machines virtuelles pour l'exécution et l'établissement de la communication TCP serveur-client.[10]

Il faut noter que ce programme utilisera principalement des appels `system()` de commande de `virtualbox` pour gérer les différentes tâches.[2]

5.4.1 Démarrage des machines virtuelles :

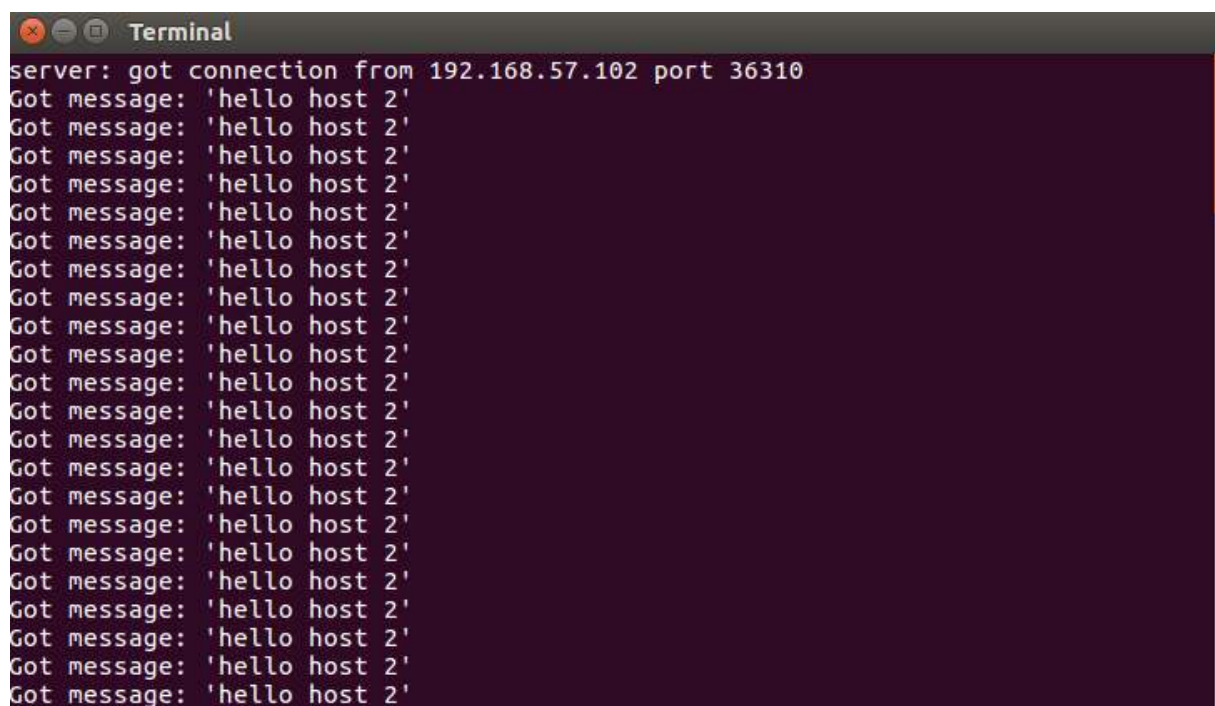
Le programme principal doit démarrer et réinitialiser les deux machines virtuelles en parallèle, en mode Headless, c'est-à-dire que les deux machines virtuelles ne montreront pas l'interface de bureau, mais elles seront démarrées en arrière-plan. Les commandes pour démarrer et réinitialiser les machines virtuelles sont respectivement les suivantes :

```
VBoxHeadless --startvm <uuid|name>
```

```
VBoxManage snapshot <uuid|vmname>
```

5.4.2 Envoi du fichier python :

Après démarrage, la machine hôte doit envoyer le fichier python qui va être exécuté dans la machine réelle. Ce fichier python contient le code pour démarrer le côté client de socket TCP et le code qui contient le calcul que la machine virtuelle devrait effectuer puis envoyer à la machine réelle. Pour envoyer ce fichier, on utilise la commande `scp` (secure copy) qui permet de copier un fichier ou un répertoire de client vers un serveur ou inversement, il est nécessaire d'installer le protocole de communication sécurisé `ssh` sur la machine hôte et la machine

A terminal window titled "Terminal" with a dark background and light text. It displays a series of log messages from a server. The first line indicates a connection from 192.168.57.102 on port 36310. Subsequent lines show the server receiving "hello host 2" messages repeatedly.

```
server: got connection from 192.168.57.102 port 36310
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
Got message: 'hello host 2'
```

Figure 6 : Côté serveur dans le deuxième réseau

La figure 8 montre le démarrage et la réinitialisation de la machine virtuelle puis l'exécution de thread pour démarrer le côté serveur. Puis la machine réelle demande à l'utilisateur d'entrée le mot de passe de l'administrateur de la machine virtuelle pour pouvoir envoyer le fichier python. Après envoi des fichiers, le programme principal lance deux terminaux des machines virtuelles dans la machine réelle par la méthode `remotedesktopxi (rdp)` pour montrer le travail effectué par la machine réelle.

Chaque machine virtuelle démarre le programme de détection du fichier python. Une fois le fichier détecté, elle l'exécute et ce fichier démarre le côté client de socket TCP et envoie l'exécution à la machine réelle (figure 9 et 10). La machine réelle puis affiche ce qui était reçu « hello host 1 ou 2 ».


```

admin@admin-MacBookPro: ~/s2project/src/projet s2
admin@admin-MacBookPro:~/s2project/src/projet s2$ ./main hello1.py hello2.py
Restoring snapshot a6a1f7e3-182f-47b4-9288-baeaaf455c4d
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
Restoring snapshot 7b440d12-50d5-425e-9588-aa74f3e79b46
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
Oracle VM VirtualBox Headless Interface 5.0.18_Ubuntu
(C) 2008-2016 Oracle Corporation
All rights reserved.

Oracle VM VirtualBox Headless Interface 5.0.18_Ubuntu
(C) 2008-2016 Oracle Corporation
All rights reserved.

starting server side communication
starting server side communication
vm1@192.168.56.101's password:
hello1.py                               100% 461      0.5KB/s   00:00
vm1@192.168.57.102's password:
hello2.py                               100% 461      0.5KB/s   00:00
admin@admin-MacBookPro:~/s2project/src/projet s2$

```

Figure 8 : lancement de programme principal

```

vm1@vm1-VirtualBox: ~
vm1@192.168.56.101's password:
Welcome to Ubuntu 16.04 LTS (GNU/Linux 4.4.0-21-generic x86_64)

 * Documentation:  https://help.ubuntu.com/

168 packages can be updated.
49 updates are security updates.

Last login: Wed Jun  1 14:52:10 2016 from 192.168.56.1
vm1@vm1-VirtualBox:~$ file detected
executing python file

```

Figure 9 : rdp de la première machine virtuelle.

```

vm1@vm1-VirtualBox: ~
vm1@192.168.57.102's password:
Welcome to Ubuntu 16.04 LTS (GNU/Linux 4.4.0-21-generic x86_64)

 * Documentation:  https://help.ubuntu.com/

168 packages can be updated.
49 updates are security updates.

Last login: Wed Jun  1 14:52:10 2016 from 192.168.56.1
vm1@vm1-VirtualBox:~$ file detected
executing python file

```

Figure 10 : rdp de la deuxième machine virtuelle.

Nous venons de préciser en détail ce qui constitue la solution envisagée en 3. Nous allons désormais vérifier qu'elle est plus efficace que l'ancienne et exposerons les limites de cette solution.

Ecrit par Amine Echraibi
Relecture : Lotfi Moubakir

6. PORTEES ET LIMITES DE LA SECURISATION

6.1 COMPARAISON ENTRE L'ANCIEN SYSTEME ET LE NOUVEAU

Il est clair qu'en utilisant la méthode d'isolation avec les machines virtuelles et les communications sécurisées en séparant les deux réseaux. Toute méthode de triche triviale utilisée par les joueurs dans l'ancien système (kill de processuss, saturation de la mémoire ...etc) s'avère inutile car le code fourni par chaque joueur s'exécute dans un environnement contrôlé, et isolé de l'environnement de l'autre joueur ce qui rend l'interférence avec le code de l'autre joueur impossible ou extrêmement difficile.

6.2 LIMITES DU SYSTEME DEVELOPPE

Le système envisagé dans le cadre de notre projet est puissant mais n'est pas impénétrable, même si l'exécution de programmes se passe en environnement contrôlé et isolé, il reste plein de méthodes pour pirater cette architecture. Dans le cadre de notre projet après finalisation de la plate-forme, on effectuera une période de tests pour essayer de détecter les failles et les vulnérabilités notre système. Voici ce que l'on essaiera de faire pour tester notre système :

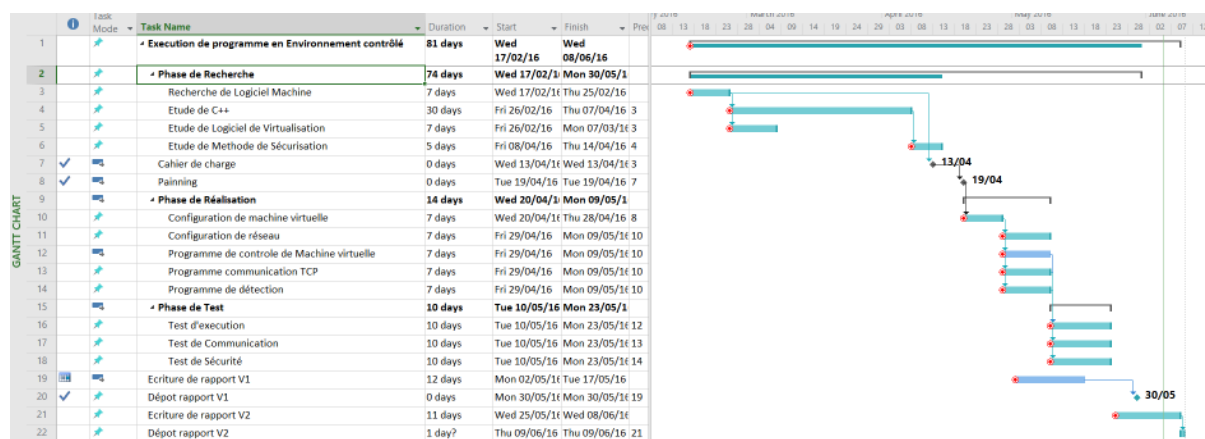
- se faire passer pour l'autre utilisateur et envoyer des données erronées sur son socket TCP, en essayant de créer une interface réseau pour se connecter à l'autre réseau.
- essayer d'augmenter les ressources de la machine virtuelle en éliminant des processus non primordiaux et par suite avoir plus de ressources que l'autre joueur (plus de RAM par exemple).

Ecrit par Lotfi Moubakir
Relecture : Amine Echraibi

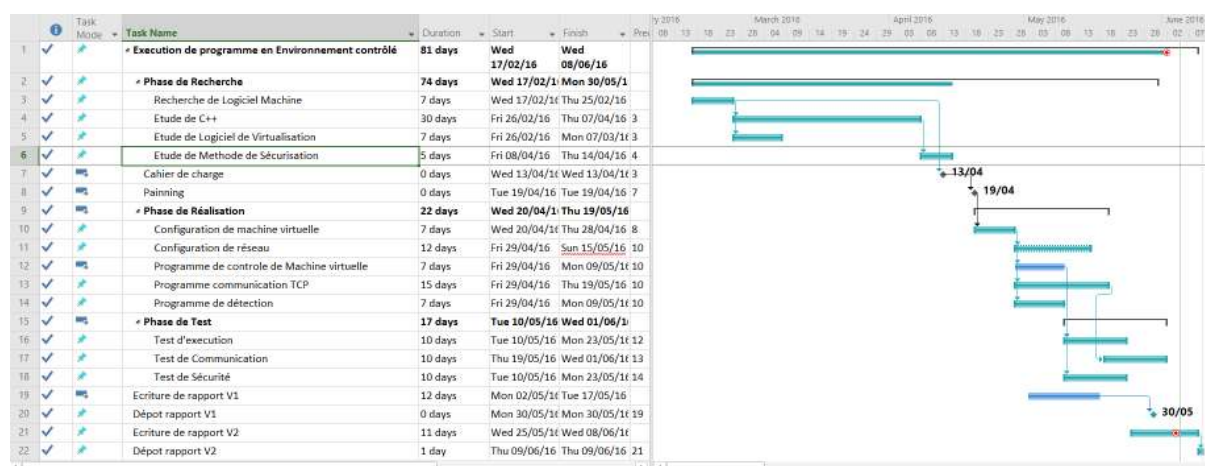
7. DIAGRAMME DE GANTT

7.1 DIAGRAMME DE GANTT

7.1.1 Diagramme de Gantt initial :



7.1.2 Diagramme de Gant final :



7.2 COMPARAISON ENTRE DIAGRAMME DE GANT PREVISIONNEL ET LE DEROULEMENT DU PROJET

On remarque d'après les diagramme de gant que la quasi-totalité du projet s'est temporairement bien déroulé, à l'exception de deux tâches, à savoir la configuration du réseau et la création du programme de communication TCP. Ceci était dû à de mauvaises estimations de temps. Pour la partie configuration du réseau, nous nous sommes rendus compte qu'il y avait beaucoup de modes de connexions et qu'il fallait faire la sélection du mode pertinent à notre projet. Nous avons aussi commencé à mettre au point un programme de communication UDP au lieu de TCP, mais nous nous sommes rendus compte que le TCP était bien plus adapté parce que nous devons assurer une qualité de service fiable.

Ecrit par Amine Echraibi
Relecture : Louis Gildas

8. CONCLUSION

En conclusion, dans le cadre de notre projet, nous avons pu créer un système de sécurisation en environnement contrôlé de la plate-forme Pyrat en utilisant l'isolation des machines virtuelles. Cette sécurisation s'est faite en séparant les deux machines virtuelles dans des réseaux différents, réduisant de manière considérable les méthodes de triches dans la plate-forme, surtout ceux qui nécessitent l'interaction directe entre le programme du joueur et la machine réelle. Cela se révèle presque impossible à cause de l'environnement contrôlé qu'on a créé.

Ce projet nous a permis de nous familiariser avec un langage de programmation qu'est le langage C++ ainsi que de découvrir certains logiciels de machine virtuelle. En utilisant VirtualBox, nous avons pu manier ce logiciel et nous intéresser aux différentes formes de communications possibles et de type de communication entre les VMS et la machine hôte (sockets).

La sécurisation que nous avons établie par virtualisation est une solution intéressante mais pas impénétrable, comme montrée dans les limites. Un des joueurs peut prétendre être l'autre joueur en créant une autre interface capable de se connecter à l'autre réseau. Ce genre d'attaque pourrait être l'objet de futur perspective concernant notre projet.

Ecrit par Louis Gildas
Relecture : Lotfi Moubakir

9. GLOSSAIRE

Adresse MAC : c'est un identifiant physique stocké dans chaque carte réseau

DHCP : c'est un protocole réseau dont le rôle est d'assurer la configuration automatique des paramètres IP d'une station, notamment en lui affectant automatiquement une adresse IP

Ethernet : c'est un protocole de commutation de paquets du réseau local

Headless : c'est une commande qui permet lors du démarrage de la machine virtuelle d'être en arrière plan invisible à l'œil de l'utilisateur

IP : c'est un identifiant attribué à chaque matériel connecté à un réseau informatique via le protocole internet

Kill : c'est une commande du système d'exploitation linux qui permet de tuer des processus tournant sur la machine

LAN (local area network) : c'est un réseau informatique local qui relie des ordinateurs dans un périmètre relativement petit.

NAT (Network Address Translation) : il fait la relation entre des adresses privées non routables et des adresses IP publiques routables.

RAM (Random Access Memory) : c'est la mémoire où l'ordinateur sauvegarde des données lors de leurs traitements.

Protocole : c'est une spécification de plusieurs règles mises en œuvre sur un réseau informatique.

Remotedesktop (rdp): c'est un protocole qui permet à un utilisateur de se connecter sur un serveur

Socket : c'est une interface de connexion permettant de relier deux machines par leurs adresses IP

Ssh : c'est un protocole de connexion à un ordinateur distant d'une manière sécurisée et chiffrée

TCP (Transmission control protocole) : c'est un protocole de transmission fiable en mode connecté

Threads : c'est un fil d'exécution représentant l'exécution d'ensembles d'instructions d'un processeur.

Pipe : les shells de système d'exploitation Linux dispose d'un mécanisme appelé pipe ou tube, ce pipe permet d'enchaîner des processus c'est-à-dire la sortie d'un processus enchaîne directement l'entrée d'un autre.

10. REFERENCES

- [1] Bastien Padeloup, *“Pyrat le rat de labyrinthe “*, Disponible sur : <http://formations.telecom-bretagne.eu/pyrat/>
- [2] Oracle Corporation, *“VirtualBox”*, Disponible sur : <https://www.virtualbox.org/manual/UserManual.html> (consulté tout au long de projet)
- [3] Korben. *“Virtualisation.”* Disponible sur : <http://wiki.korben.info/virtualisation> (15 Mars 2016)
- [4] Robert Lafore *“Object-Oriented Programming in C++”, 4ème édition, les États-unis, December 2001, 981 pages, ISBN :0-672-32308-7.(consulté tout au long de projet)*
- [5] Auteur inconnu. *“Tunneling Viruses.”* Disponible sur : <http://www.cknow.com/vtutor/vttunneling.html> (15 Mars 2016)
- [6] Auteur inconnu, *“Network Sockets”*, Disponible sur : https://en.wikipedia.org/wiki/Network_socket (24 avril – 30 avril 2016)
- [7] Prentice Hall *“sockets tutorial”*, Disponible sur : http://www.linuxhowtos.org/C_C++/socket.htm (tout au long du projet)
- [8] VMware corporation ,*“Selecting IP Addresses on a Host-only Network or NAT Configuration“*, Disponible sur : https://www.vmware.com/support/ws55/doc/ws_net_advanced_ipaddresses.html (fn avril ddbut mai 2016)
- [9] Gabon, Timothy. *“ Security ROI - Payback for Comprehensive Protection.”* Disponible sur : http://www.fortune.com/sitelets/sections/fortune/tech/2002_07security.html (15 Mars 2016)
- [10] Justin Seitz *“Black hat python”, 1ère édition, les États-unis, December 2014, 163 pages, ISBN :1-59327-590-0*

w w w . t e l e c o m - b r e t a g n e . e u

Campus de Brest

Technopôle Brest-Iroise
CS 83818
29238 Brest Cedex 3
France
Tél. : + 33 (0)2 29 00 11 11
Fax : + 33 (0)2 29 00 10 00

Campus de Rennes

2, rue de la Châtaigneraie
CS 17607
35576 Cesson Sévigné Cedex
France
Tél. : + 33 (0)2 99 12 70 00
Fax : + 33 (0)2 99 12 70 19

Campus de Toulouse

10, avenue Edouard Belin
BP 44004
31028 Toulouse Cedex 04
France
Tél. : +33 (0)5 61 33 83 65
Fax : +33 (0)5 61 33 83 75

